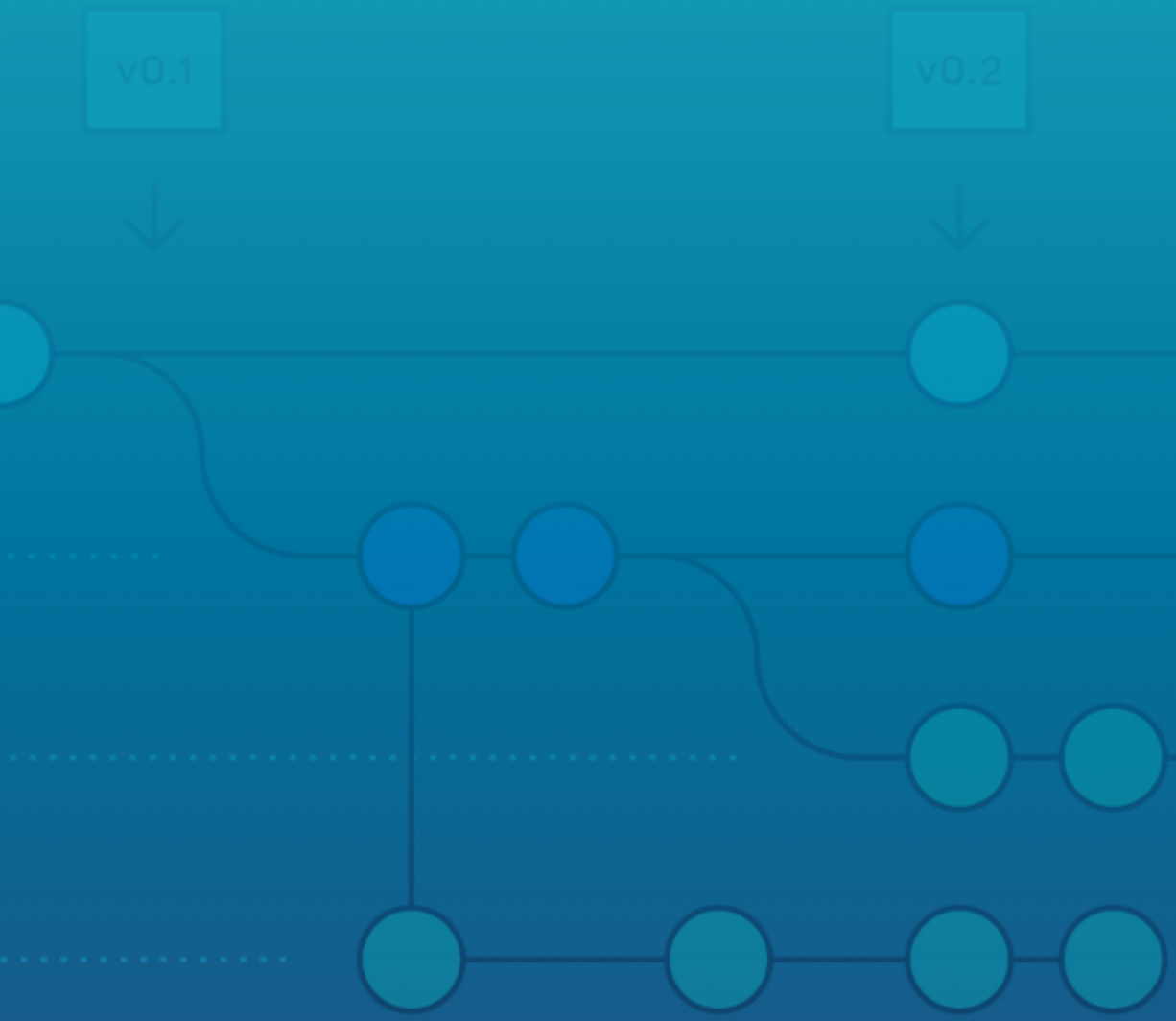


Git & Source Control

Meer Ushter

Iguana User Conference 2023

Overview

Introduction to Source Control

- What is Source Control
- What is Version Control System

Iguana Source Control

- Why Iguana Needs Source Control
- How does Iguana Apply Source Control
- Iguana Supported Git Repositories

Source Control Solutions

- Deployment Automation

iNTERFACEWARE | integration made easy

What is Source Control?

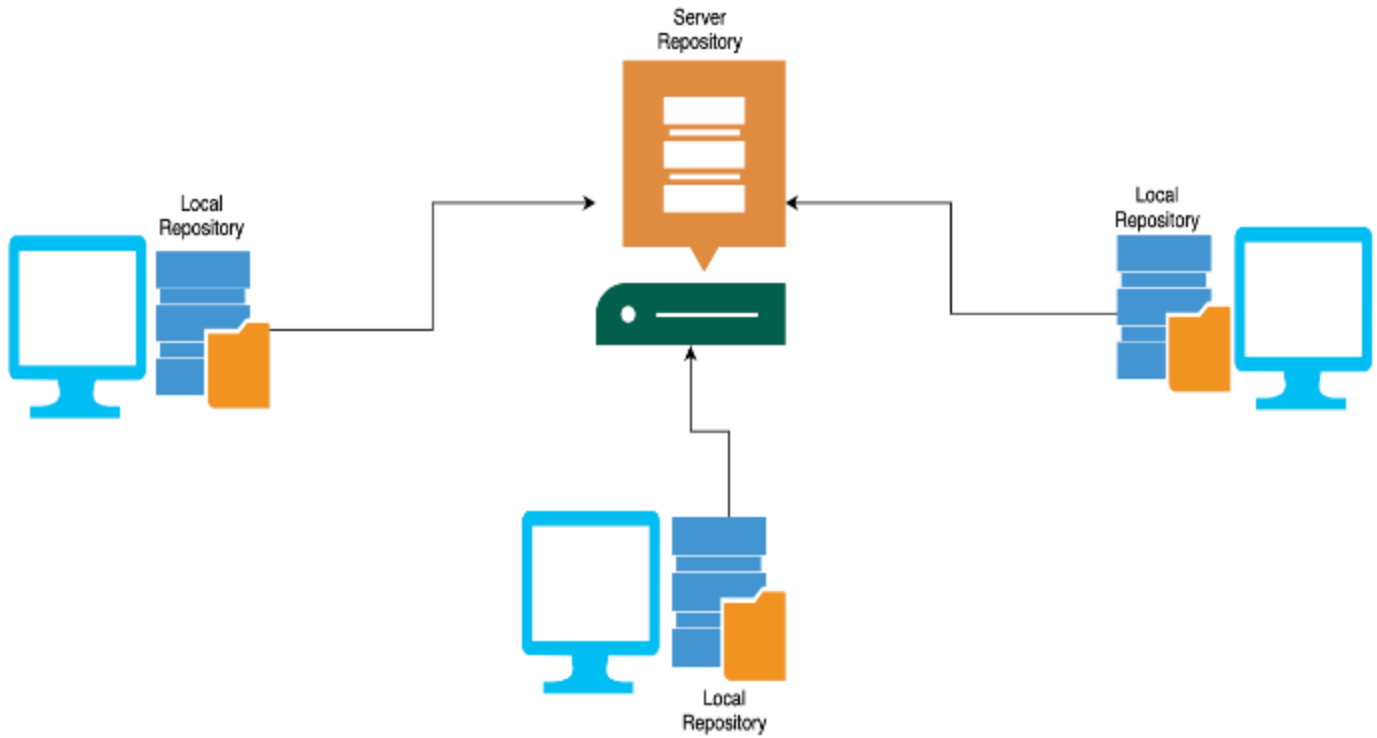
A practise in order to keep track and manage changes of code

Be able to Backup and Restore

Allows Collaboration Between Developers

INTERFACEWARE | integration made easy

Version Control System (VCS)



VCS is a Software Tool

Git and Mercurial are part of VCS

VCS uses local and shared repository

INTERFACEWARE | integration made easy

Iguana Source Control

iNTERFACEWARE | integration made easy

Why does Iguana need Source Control?

- Save And Backup
- Document Code Changes and User Information
- Ability to Revert Changes
- Share Iguana Projects

What Iguana Source Control Offers



Iguana History

Translator Commit History

Import/Export Channels



Iguana Repository Options

Local repository

A folder on the Iguana host

Example path:
/path/to/repository

Local Repository

- Hosted on a local machine
- Able to access offline
- Only accessible by internal server users

Remote repository

Shared through a Git hosting service

Example:
https://github.com/example/project.git

Remote Repository

- Hosted on a server
- Needs internet access
- Accessible by external users

Connecting Iguana with Remote Repository

Iguana Supported Repo (local, remote)

- Azure, Github, BitBucket, GitLab and more

Common Connection Methods

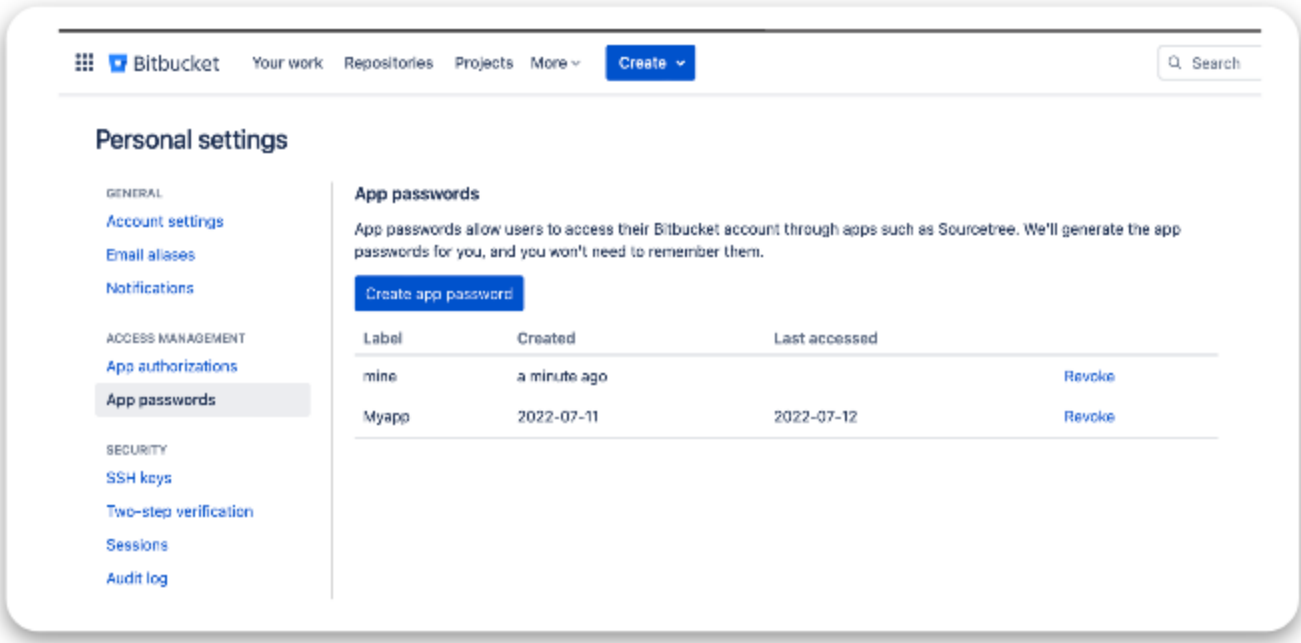
- App password
- SSH

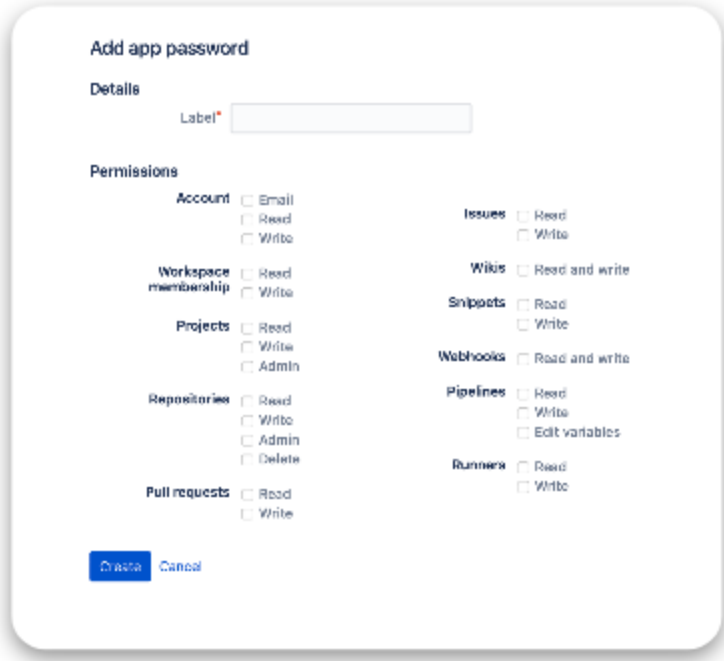


Connecting Iguana with App Passwords

App Passwords

- User Based-Tokens
- Allows Control of Permissions
- Generate unique passwords for individual users



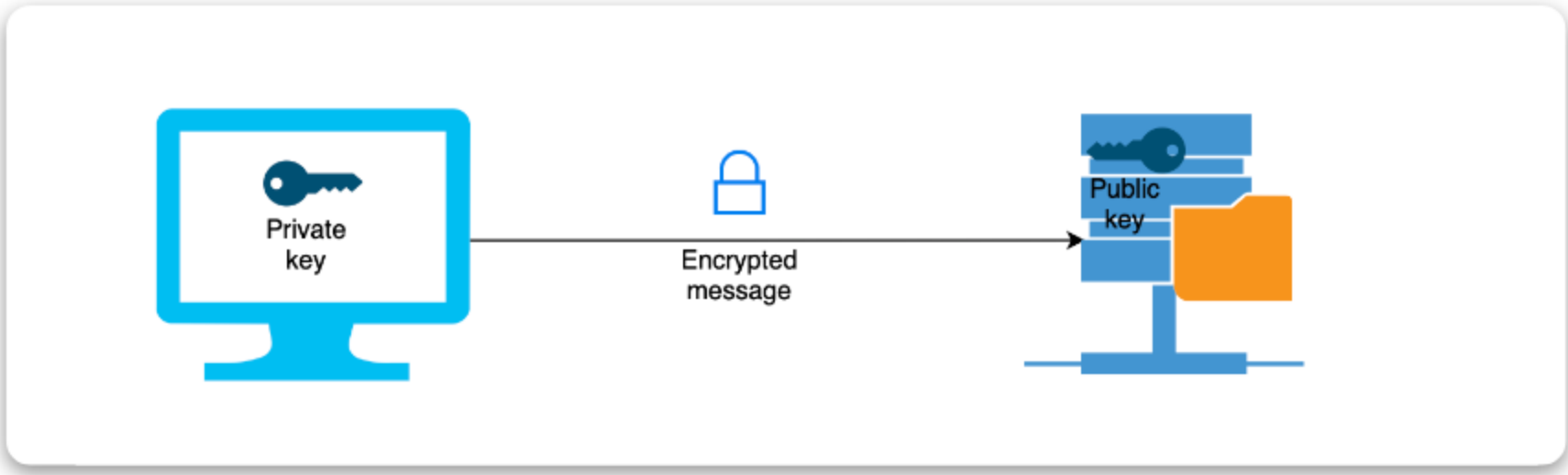


INTERFACEWARE | integration made easy

Connecting with Iguana using SSH Key

SSH Keys

- Encrypted key files
- Provides Secure connection over the public internet
- Stores the public key in the remote repository
- Stores private key on the local machine



Pros & Cons

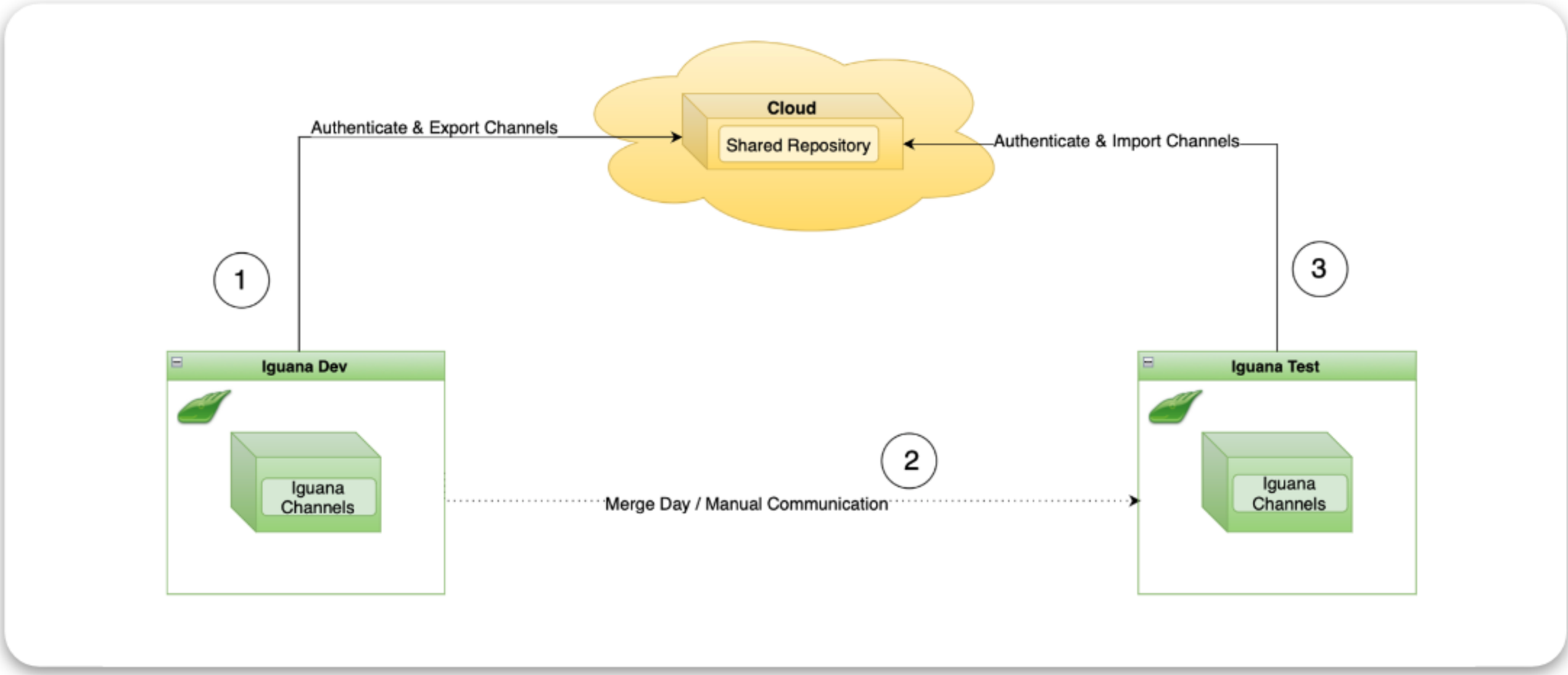
	App Password	SSH Key
Pros	• Customized permission for each user • Documents user's name who accessed the repository (push/pull)	• Does not Require user to enter credentials • Same key files can access multiple repositories
Cons	• Requires to physically save the password token	• Display username of who generated the key files

INTERFACEWARE | integration made easy

Source Control Solutions

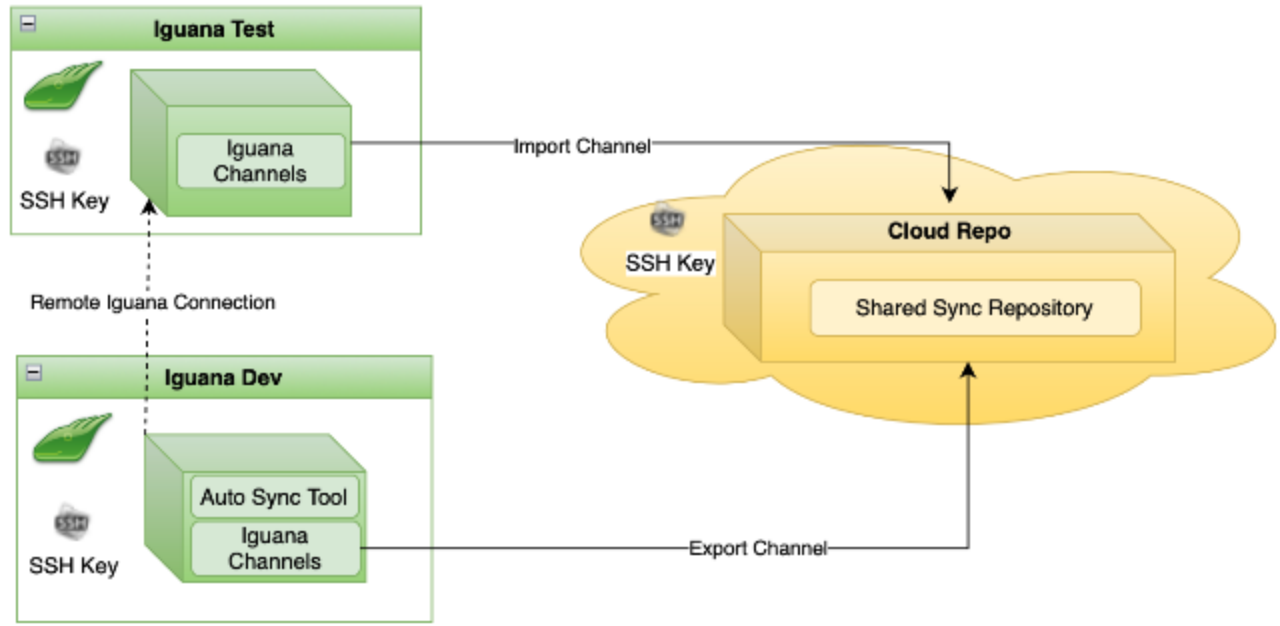
INTERFACEWARE | integration made easy

Manual Process



iNTERFACEWARE | integration made easy

Automated Solution



Automated Solution (Detecting Changes)

```
local MarkerString = "Channel configuration modified: "
-- Get channel updates since last sync
local Status, Logs = pcall(Servers.this.apiCall,Servers.this,net.http.get, {
      url  = Servers.this.url .. 'api_query',
      parameters={
         type    = 'message',
         reverse = 'true',
         filter  = MarkerString,
         after   = LastSync,
      },
      live  = true,
      cache_time = conf.cache_time,
      timeout = conf.extended_timeout,
   })
```



Result:
Returns all Iguana Logs as XML

Parameters:
type : Returns messages of selected type
filter : Returns messages that includes a specified string value
reverse: Returns messages in order from newest to oldest

iNTERFACEWARE | integration made easy

Deployment Automation (view local channels)

```
local exportList = net.http.get{
   url = baseUrl .. '/sc/view_local_channels',
   auth = configuredCredentials,
   parameters = {for_export = true},
   live = true
}
```

Result:
Returns a list of channels in Iguana in a JSON format, matching the specified parameters.

Parameters:
for_export : true/false, false will return all channels even the ones with no commits

iNTERFACEWARE | integration made easy

Deployment Automation (export channels)

```
local result = net.http.post{
   url = baseUrl .. '/export_channels',
   auth = configuredCredentials,
   parameters = exportParameters,
   live = true
}

-- Populate exportParameters with the
-- channel list and collision confirmations.
```

Returns:
Summary of the request and confirmation of success

Parameters
sc_task: Specify task “export_channels”
commit_message: Specify commit message
remote_name: Remote Repository name
export_list : List of channels in local for export
channel_collision_confirmation: Required to resolve channel collisions
overwrite_shared_file_confirmation: Required to resolve shared file collisions

iNTERFACEWARE | integration made easy

Deployment Automation (view remote channels)

```
local importList = net.http.get, {
   url = 'https://localhost:6543/sc/view_remote_channels',
    auth = conf.auth,
   parameters = {
      ["remote_name"] = "Remote Repo"
   },
}
```

Returns:
List of channels to import from your remote repository

Parameters:
remote_name: Name of the remote repository from Iguana's Repository list

Deployment Automation (import channels)

```
local result = net.http.post{
   url = baseUrl .. '/import_channels',
   auth = configuredCredentials,
   parameters = importParameters,
   live = true
}

-- Populate importParameters with the
-- remote channel list and confirmations.
```

Returns:
Summary of the request and confirmation of success

Parameters
sc_task: Specify task import_channels
commit_message: Specify commit message
remote_name: Remote Repository name
channel_import_list : List of channels from the remote repo to
channel_version : Specify channel version , 6_1_0 or 6_0_0
channel_collision_confirmation:
Required to resolve channel collisions
overwrite_shared_file_confirmation:
Required to resolve shared file collisions

iNTERFACEWARE | integration made easy



Thank you

Questions?